Университет ИТМО

Техническое задание

Calorica

Мобильное приложение для учёта питания
с серверной частью

Семестровый MVP
Третий этап учебного проекта
Версия 1.7 · 23 октября 2026 года

Команда проекта и формальные роли

Валера — DevOps-инженер
Камила — тестировщик
Богдан и Николай — разработчики
Рауль и Мария — аналитики

2026

1 Назначение и границы

Calorica — мобильное приложение для учёта питания, калорий, белков, жиров и углеводов с серверной частью. Оно предназначено для человека, которому нужно видеть свой рацион без ручного подсчёта показателей. КБЖУ означает калории, белки, жиры и углеводы; MVP — минимальная работоспособная версия.

Результат семестра — Android-приложение, в котором пользователь входит, выбирает или создаёт продукт, записывает съеденное и видит дневные КБЖУ. Сервер хранит данные, проверяет права и выполняет расчёты.

Объём поставки

- В Android два раздела: «Продукты» и «Дневник». Состав вводится процентами, съеденная порция — граммами. Дневной итог представлен числовой сводкой.
- На сервере сохраняется целевая модель: вложенные составы, коэффициенты выхода, конвертация единиц, лейблы, права доступа и два режима мягкого удаления. Дополнительные операции сервера проверяются через API — программный интерфейс.
- В поставку входят подписанный установочный APK, развёрнутый сервер, исходный код, миграции БД, описание API, инструкции и результаты приёмочных проверок.

Основание и команда

ТЗ объединяет требования второго этапа и архитектуру третьего этапа учебного проекта: технологии, компоненты, модель данных и взаимодействие системы.

Формальные роли: Валера — DevOps-инженер; Камила — тестировщик; Богдан и Николай — разработчики; Рауль и Мария — аналитики. Все участники могут работать с агентами.

2 Функции Android приложения

SM-01. Аккаунт. Регистрация по email и паролю, вход, восстановление сессии при повторном запуске и выход. Для регистрации достаточно email и пароля.

SM-02. Каталог. Общий список базовых и собственных продуктов, поиск по названию и карточка с актуальными КБЖУ на 100 г. Начальный каталог содержит около 50 продуктов. Поиск нормализует регистр, «ё/е», «ъ/ь»; точные и префиксные совпадения выше похожих. Для короткого запроса используется префикс, для опечаток — триграммный поиск.

SM-03. Личный продукт. Создание и изменение названия и ручных КБЖУ на 100 г. Удаление мягкое: карточка исчезает из выбора, но сохраняется в составах и истории. Базовые продукты в Android только просматриваются.

SM-04. Состав. Выбор компонентов, включая составные продукты, ввод и изменение процентов с суммой ровно 100%. Точные доли сохраняются при повторном открытии. Добавление компонентов превращает продукт в составной; удаление всех компонентов возвращает его к конечному с валидными ручными КБЖУ.

SM-05. Выход блюда. Необязательный общий коэффициент в дополнительном блоке; без настройки выход равен 1. Если через API заданы коэффициенты ингредиентов, общий выход только показывается. Правка формы не очищает скрытые настройки.

SM-06. Добавление питания. Выбор дня, по умолчанию сегодняшнего, продукта, положительной съеденной массы в граммах и времени с автоматическим предзаполнением. Можно выбрать несколько лейблов: «завтрак», «обед», «ужин», «перекус». Масса всего рецепта не подставляется вместо съеденной порции.

SM-07. Правка питания. Изменение продукта, количества, даты, времени и лейблов; удаление записи без удаления карточки продукта. После изменений данные обновляются.

SM-08. Итог дня. Над дневником показываются суммарные калории и БЖУ за выбранный локальный день. Изменение карточки ингредиента влияет и на показатели старых записей.

SM-09. Состояния интерфейса. Загрузка, пустой список и ошибка различаются; доступен повтор загрузки. Сохранение блокирует повторный запрос по двойному нажатию. При потере ответа приложение не утверждает, что запись не сохранилась, и не повторяет её автоматически. Нужна проверка результата повторной загрузкой.

3 Функции сервера и правила данных

SB-01. Граф продуктов. Единая сущность продукта и связи с исходными массовыми долями. Вложенность разрешена, прямые и косвенные циклы запрещены. Общий ингредиент учитывается по каждому пути. Личный состав использует только свои и базовые продукты; базовый — только базовые. Ручные КБЖУ и лейблы родителя сохраняются, но игнорируются до возврата в конечное состояние.

SB-02. Состав и выход. API принимает либо граммы ингредиентов, либо проценты. Массы преобразуются в доли; неверная сумма процентов отклоняется. Хранятся доли и заданные коэффициенты, не массы рецептуры. Если задан выход хотя бы одной связи, выход блюда равен взвешенной по исходным долям сумме выходов ингредиентов; пропущенные значения в расчёте равны 1. Иначе применяется общий выход или 1. Явная 1 отличается от отсутствия. Одновременное задание двух режимов запрещено.

КБЖУ на 100 г состава — взвешенная сумма актуальных КБЖУ компонентов, делённая на выход блюда. Коэффициент связи повторно не применяется к нутриентам. Общий выход можно вычислить из готовой массы только при вводе состава граммами. Расчётный выход из связей не сохраняется. Смена режимов атомарна; старый общий выход после очистки связей скрыто не восстанавливается.

SB-03. Дневник и единицы. Хранятся продукт, UTC-время, лейблы и съеденные граммы, без снимка КБЖУ и единицы ввода. API конвертирует миллилитры, только если для продукта задан коэффициент. Личные продукты изначально поддерживают только граммы.

SB-04. Лейблы. Продукт может иметь несколько общих лейблов. Для составов распределяются граммы готовой порции до листьев: по готовым долям при выходе на связях и по исходным долям при общем выходе. Пересечения лейблов дают повторный вклад в разные группы. Удаление лейбла удаляет его связи.

SB-05. Управление и удаление. Admin управляет базовыми продуктами и общими продуктовыми лейблами через API, но не чужими личными данными. API поддерживает сохранение удалённого продукта в составах или мягкий каскад к зависимым родителям, с предпросмотром и повторной проверкой прав. Дочерние ингредиенты и дневник не удаляются. При чужих личных зависимостях каскад Admin недоступен.

SB-06. Серверная аналитика. КБЖУ рассчитываются при запросе за день, неделю пн–вс или полный календарный месяц по выбранной дате и IANA-поясу клиента. Среднее включает пустые дни. Несколько пищевых лейблов повторяют вклад по группам, но не удваивают общий итог. Постоянные рассчитанные итоги не хранятся.

4 Нефункциональные требования

SN-01. Безопасность. HTTPS, проверка прав сервером, хеширование паролей Argon2id, короткий access token и ротируемый refresh token. Выход отзывает текущую сессию. Секреты отсутствуют в Git и логах; refresh token Android защищён ключом Keystore. Проверка: вход/выход, повтор refresh и запросы к чужим данным должны обрабатываться по правилам доступа; auth-запросы ограничены по частоте.

SN-02. Целостность. Состав и выход сохраняются одной транзакцией; граф меняется под блокировкой владельца. Проверки циклов, прав и гонок удаления выполняются интеграционными тестами на PostgreSQL. Ошибка не оставляет частичных изменений. Блокировка одного владельца не заменяет протокол удаления базовых зависимостей.

SN-03. Точность. NUMERIC/BigDecimal, масса до 3 знаков, ручные КБЖУ до 2, доли и заданные коэффициенты до 12. Сумма нормализованных долей ровно 1; экранное округление не изменяет данные. Проверка: равные массы, вложенные составы, NULL и 1, контрольные числовые примеры; неверные диапазоны и бесконечности отклоняются.

SN-04. Сохранность. Записи переживают перезапуск. Автоматический разрушительный откат БД запрещён.

SN-05. Сеть. Каждый запрос имеет конечный таймаут; в первой реализации 10 секунд. После таймаута UI показывает ошибку и позволяет повторить загрузку. Неопределённая запись автоматически не повторяется. Проверка: отключение сети и задержка ответа. Загрузка и сохранение данных выполняются через серверное API.

SN-06. Совместимость и удобство. Android API 26 и выше, русский интерфейс, стандартные компоненты, сенсорные цели не меньше 48 dp. Основной сценарий проверяется на одном эмуляторе и одном доступном физическом устройстве; версии фиксируются. Локальные дата и время проверяются около полуночи в двух часовых поясах.

SN-07. Выпуск. Независимые проверки и автоматические выпуски Android и сервера после успешных проверок в main. APK подписан, сервер развёрнут по точному digest образа с проверкой готовности. Документационные изменения не запускают релизы. Проверка: изменения только Android, только сервера и только документов.

SN-08. Воспроизводимость. Kotlin/Compose, Kotlin/Spring Boot, Spring JDBC, PostgreSQL, Liquibase; единый репозиторий. Другой участник выполняет инструкции запуска, миграции и выпуска. Миграции проверяются на пустой и предыдущей БД.

5 Процессы и план семестра

Процесс до автоматизации

Пользователь записывает еду, ищет КБЖУ и вручную пересчитывает значения на порцию. Для блюда отдельно суммирует ингредиенты. Исправление продукта требует повторного подсчёта, а дневные итоги приходится собирать самостоятельно.

Процесс с приложением

Пользователь входит, выбирает дату и продукт или создаёт состав, вводит съеденные граммы и сохраняет запись. Сервер проверяет данные и права, рассчитывает КБЖУ по актуальным карточкам; дневник показывает результат. При правке записи итог обновляется. При недопустимом составе сервер отклоняет операцию целиком; при сетевой ошибке пользователь проверяет результат повторной загрузкой.

Порядок реализации

Шаг	Результат
Контракт и каркас	Схема данных, API, сессии, сервер с БД и первый запрос Android к API
Каталог	Базовые и личные продукты, поиск и изоляция данных пользователей
Расчёты	Составы, выход, единицы, лейблы и удаление; проверки доменных правил
Android	Редактор состава, дневник, дневной итог и интеграция с сервером
Проверки и выпуск	Независимый CI/CD, подписанный APK, сервер, миграции и проверка перезапуска
Приёмка	Исправления, проверка SA-01—SA-17, демонстрация и материалы сдачи

Переход к следующему шагу опирается на готовый результат предыдущего и согласованный контракт. Первый сквозной сценарий проверяет выбранный стек и взаимодействие клиента с сервером. Независимые работы могут выполняться параллельно.

Агенты выполняют ограниченные задачи по общему контракту. Команда проверяет код, исправляет дефекты и подтверждает результаты проверок. Изменения объёма согласуются явно по результатам реализации.

6 Порядок приёмки

Приёмка проводится после реализации на зафиксированных версиях APK, сервера и БД. Для проверки записываются результат и доказательство: тест, журнал или демонстрация. Отсутствующая реализация означает «не проверено», а не успешное выполнение.

Проверки	**Что должно быть подтверждено**
SA-01 — SA-03	Установка, сессия, изоляция аккаунтов, каталог и личный продукт
SA-04 — SA-08	Точные доли, вложенность, отсутствие циклов, оба выхода и ввод масс через API
SA-09 — SA-10	Правка дневника, актуальные КБЖУ, UTC и локальная дата
SA-11 — SA-14	Два удаления, конвертация, лейблы и календарная аналитика через API
SA-15 — SA-17	Сетевые ошибки, независимый выпуск, сохранность при перезапуске

Критические примеры: два ингредиента по 50% с 400 и 80 ккал/100 г и выходами 0,5 и 1,5 дают выход блюда 1 и 240 ккал/100 г. Порция 200 г даёт 480 ккал и распределяется как 50/150 г по ингредиентам. Неделя с 700 ккал только в одном дне даёт среднее 100 ккал/день. Запись с двумя пищевыми лейблами не удваивает общий итог.

Нефункциональные требования SN-01 — SN-08 проверяются способами из раздела 4. SA-01 — SA-16 проверяются по семестровому плану приёмки; SA-17 — сохранность записей после перезапуска сервера и БД. Утечка чужих данных, неверные расчёты, потеря сохранённых записей и неработающий основной сценарий блокируют сдачу. Отклонения согласуются явно, без скрытого упрощения модели или исключения проверок.

7 Архитектура и технологии

Целевая архитектура Calorica включает Android-клиент, сервер и PostgreSQL. Сервер — модульный монолит: предметные компоненты работают в одном процессе и используют общую БД. Клиент отвечает за ввод и отображение, сервер — за права, хранение и расчёты; связанные изменения сохраняются одной транзакцией.

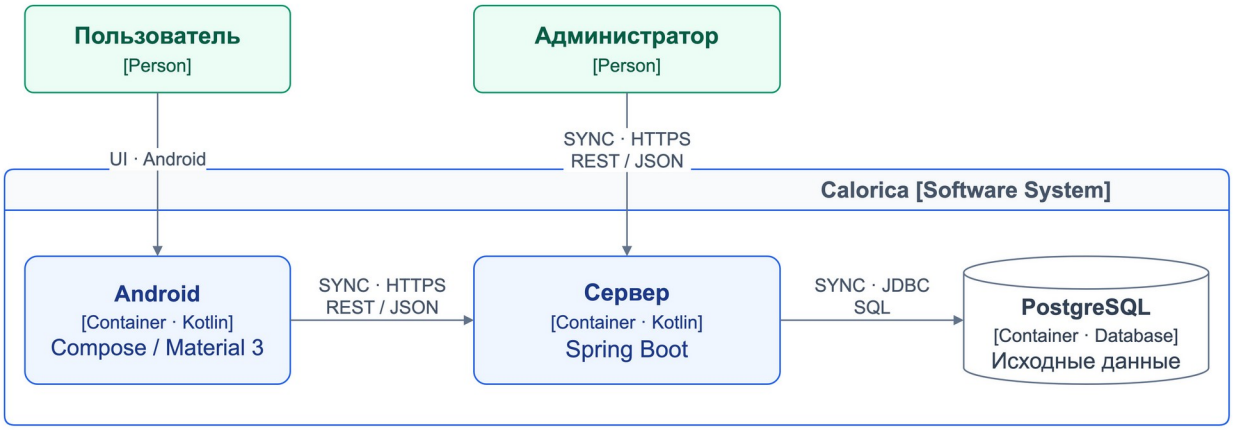
Часть	**Технологии**
Android	Kotlin, Jetpack Compose, Material 3
Клиентский слой	ViewModel, StateFlow, Coroutines, Hilt, Retrofit, OkHttp
Сервер	Kotlin, Spring Boot, Spring Security, JDK 21
Данные	PostgreSQL, Spring JDBC, явный SQL
Миграции и API	Liquibase, SQL, REST, JSON, OpenAPI
Выпуск	GitHub Actions, GitHub Release, GHCR, Docker Compose, Nginx

8 Контекст системы



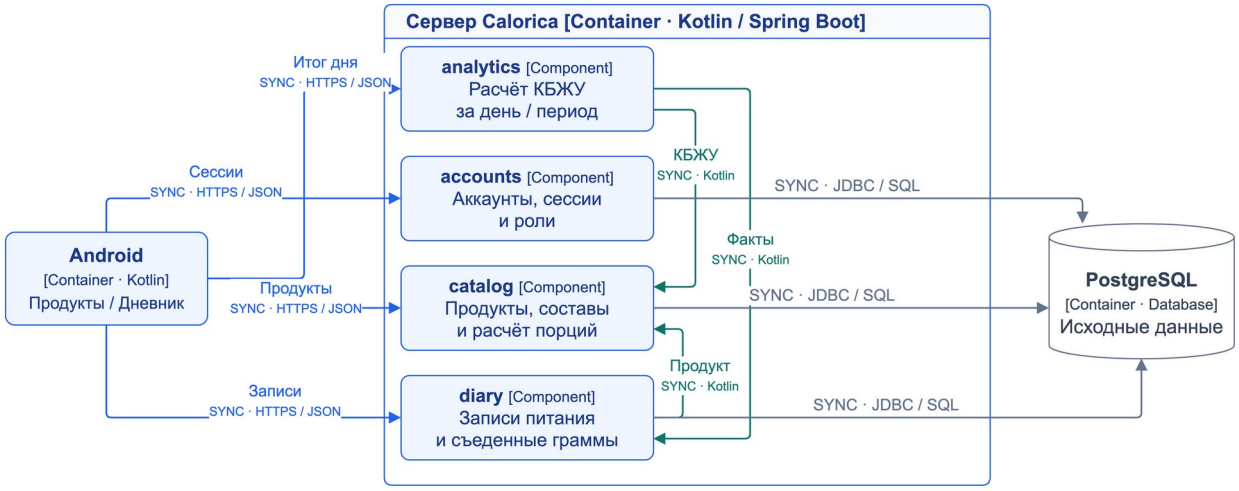
Пользователь ведёт личные продукты и дневник, получает показатели питания. Администратор обслуживает базовый каталог и общие продуктовые лейблы через API. Сервер проверяет роль и владельца данных при обращении к личным ресурсам. Расчёты выполняются внутри Calorica.

9 Контейнеры системы



Android обращается к REST API сервера по HTTPS и передаёт JSON. SYNC означает запрос с ожиданием результата или ошибки. Сервер работает с PostgreSQL через JDBC и SQL, применяя проверки доступа и транзакции к операциям с данными.

10 Компоненты серверного приложения

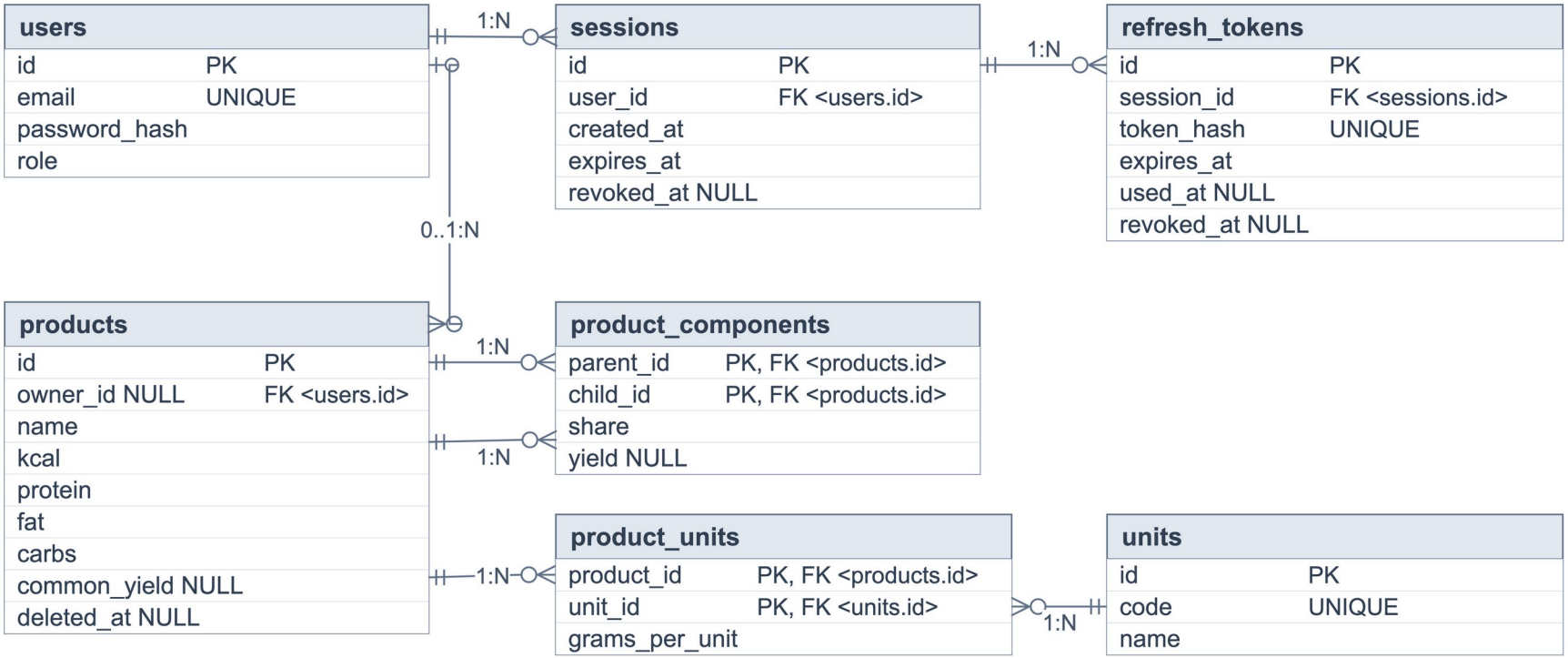


Android вызывает accounts для входа и сессий, catalog для продуктов, diary для записей и analytics для показателей питания. Межмодульные связи — синхронные вызовы Kotlin внутри одного процесса. Diary проверяет продукт через catalog, analytics получает факты через diary и использует расчёт catalog.

Accounts, catalog и diary обращаются к своим данным в PostgreSQL через Spring JDBC. Analytics вычисляет сводки при запросе, используя эти компоненты. Spring Security проверяет HTTP-доступ к соответствующим операциям.

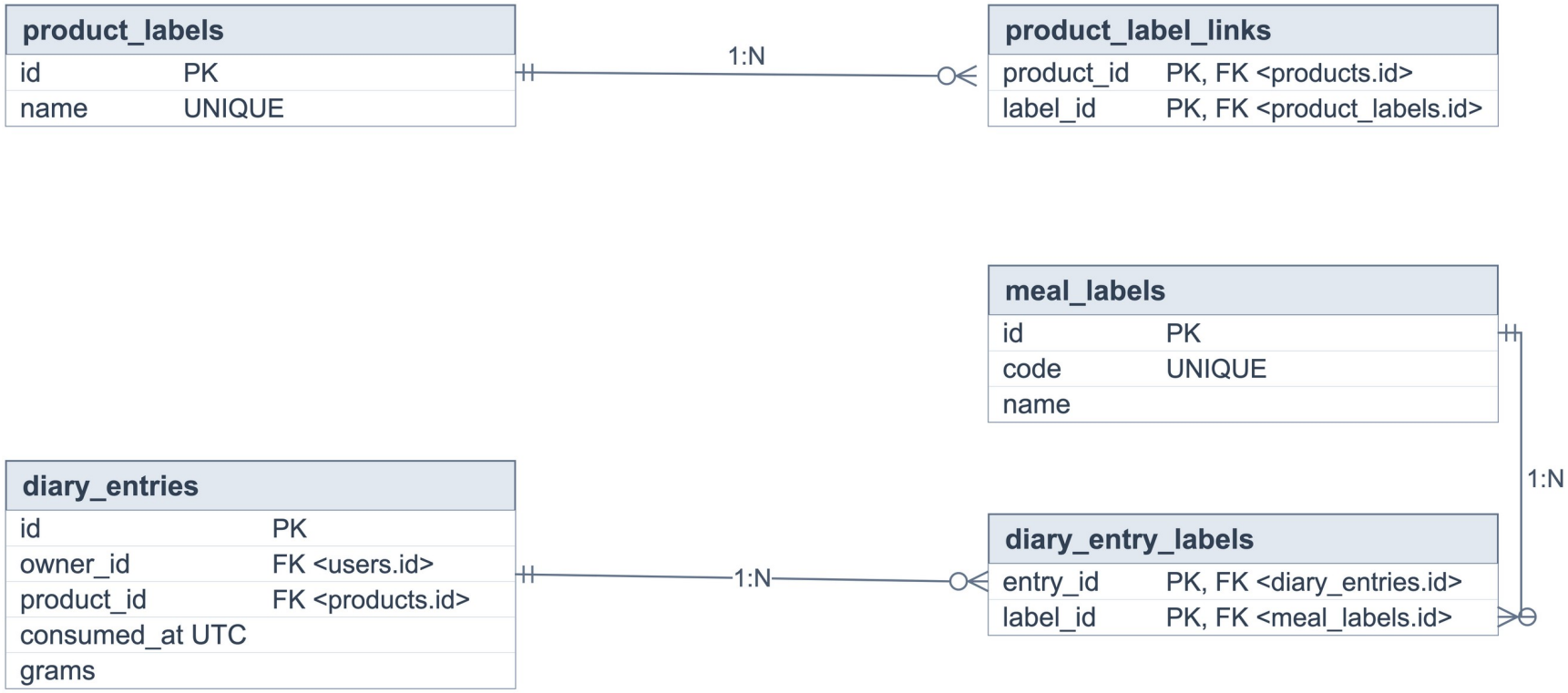
11 Логическая модель базы данных

11.1 Аккаунты и каталог



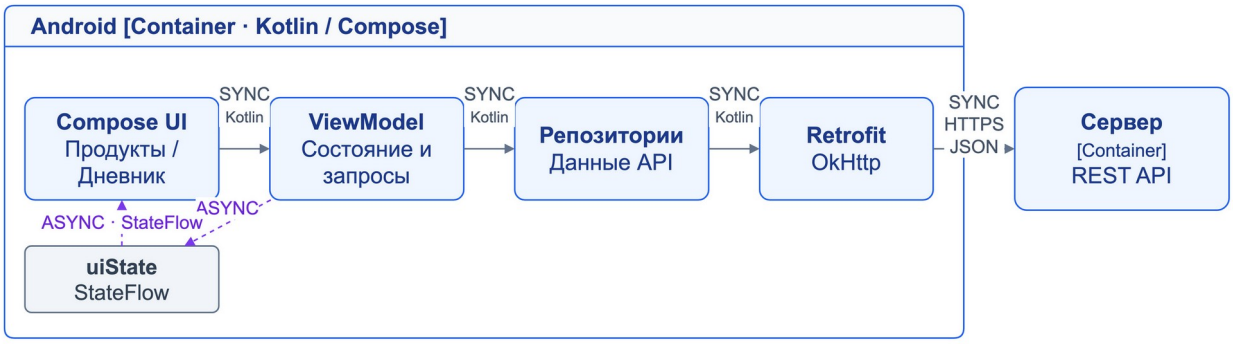
Users хранит аккаунты, sessions связывает с пользователем сессии, refresh_tokens поддерживает обновление токенов и отзыв сессии. Products содержит базовые карточки с пустым owner_id и личные продукты с владельцем. Product_components хранит связи родителя с ингредиентом, доли и коэффициенты выхода; сервер проверяет отсутствие циклов. Units задаёт единицы, product_units — конверсии для конкретного продукта.

11.2 Дневник и метки



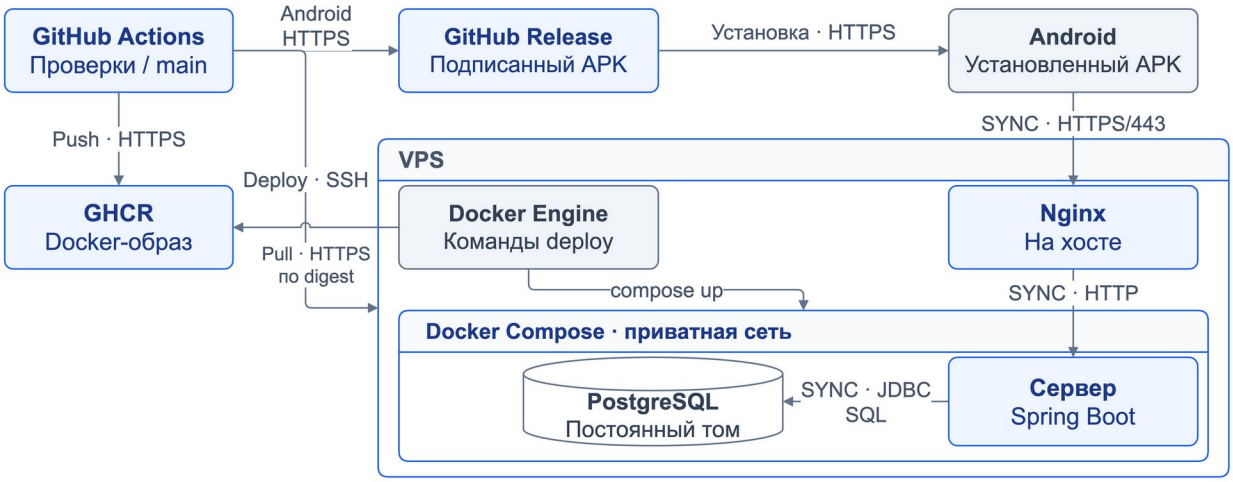
Diary_entries хранит владельца, продукт, съеденные граммы и время в UTC. Product_labels и product_label_links позволяют назначать продукту несколько общих лейблов. Meal_labels и diary_entry_labels связывают запись питания с несколькими пищевыми лейблами. КБЖУ рассчитываются при чтении по актуальным карточкам, поэтому правка ингредиента влияет и на расчёт старых записей. Мягкое удаление сохраняет ссылки из составов и дневника.

12 Краткое устройство Android клиента



Compose формирует разделы «Продукты» и «Дневник». ViewModel управляет состоянием и вызывает репозитории; Retrofit и OkHttp обращаются к API. Coroutines выполняют сетевые операции вне UI-потока. HTTP-запрос получает ответ сервера, а StateFlow асинхронно передаёт изменения состояния экрану. Hilt предоставляет зависимости слоёв.

13 Целевой выпуск и развёртывание



GitHub Actions независимо проверяет и выпускает Android и сервер из main. Подписанный APK публикуется в GitHub Release, серверный Docker-образ — в GHCR. CI подключается к VPS по SSH; Docker Engine получает образ из GHCR по HTTPS с точным digest, затем Compose запускает версию. Liquibase обновляет структуру БД, readiness подтверждает готовность сервера.

Nginx на хосте принимает HTTPS и передаёт HTTP-запросы серверу в Compose. Сервер обращается к PostgreSQL через JDBC во внутренней сети. Постоянный том сохраняет данные между перезапусками контейнера.